



الجمهورية العربية السورية

جامعة دمشق

كلية الهندسة

المعلوماتية

سنة رابعة ذكاء صناعي

البرمجة التفرعية

المهندس المشرف: حذيفة عقيل

2026-2025

الطلاب :

أحمد علي حمودة

الأيهام أمير طه

جابر يحيى شاهين

عمر محمد نعيم بيازيد

يارا منذر الحسن

Contents

3.....	Concurrent Access & Data Integrity
8.....	Control Capacity & Management Resource
9.....	Queues Asynchronous
12.....	batch processing للمبيعات اليومية
15.....	شرح موجز عن مهمة توزيع الأحمال (Load Distribution)

حماية البيانات المشتركة من التضارب

Concurrent Access & Data Integrity

ما هي مشكلة Race Condition؟

تحدث مشكلة Race Condition عندما يحاول أكثر من مستخدم في نفس الوقت تعديل نفس البيانات المشتركة، فيقرأ كل منهم نفس القيمة القديمة، ويجري حسابه عليها، ثم يحفظ النتيجة — دون أن يعلم أحدهم بتعديل الآخر. النتيجة: بيانات فاسدة.

مثال توضيحي على مشكلة البيع الزائد (Overselling):

المستخدم	العملية	قيمة المخزون المقروءة	النتيجة المحفوظة
المستخدم أ	يقرأ المخزون 5	—	—
المستخدم ب	يقرأ المخزون 5	—	—
المستخدم أ	يطرح 3 من 5	—	يحفظ: 2
المستخدم ب	يطرح 4 من 5	—	يحفظ: 1

المشكلة: المستخدم ب قرأ القيمة 5 (قبل أن يحفظ المستخدم أ تعديله)، فظن أن المخزون لا يزال 5، وهكذا تم البيع بشكل خاطئ وأصبح المخزون الفعلي 1 بدلاً من 2.

قرار اختيار قاعدة البيانات

2.1 لماذا لا تكفي SQLite لهذا المتطلب؟

كانت قاعدة البيانات الافتراضية في المشروع هي SQLite. وهي مناسبة لمرحلة التطوير الأولي، لكنها غير كافية لتطبيق قفل الصفوف (Row-Level Locking) المطلوب لمعالجة Race Condition، والسبب يعود إلى نموذج القفل المستخدم:

المعيار	SQLite	PostgreSQL
نوع القفل	قفل على مستوى الملف كاملاً	قفل على مستوى الصف المحدد

دعم select_for_update()	جزئي وغير موثوق	كامل وموثوق
التزامن الحقيقي	غير مدعوم	مدعوم عبر MVCC
مناسب لـ 100+ مستخدم	لا	نعم
متزامن		

ما هو MVCC وكيف يحل المشكلة؟

يستخدم PostgreSQL تقنية MVCC (Multi-Version Concurrency Control) وهي آلية تعطي كل معاملة (Transaction) نسخة خاصة بها من البيانات لحظة بدء التنفيذ. هذا يعني:

1. كل قراءة تتم على نسخة ثابتة لا يعدّل عليها أحد أثناء تنفيذ المعاملة.
2. الكتابة لا تحجب القراءة، والقراءة لا تحجب الكتابة — مما يزيد الأداء.
3. القفل يُطبّق فقط على الصف المحدد وليس على قاعدة البيانات كأكملها.

القرار الهندسي: تم الانتقال من SQLite إلى PostgreSQL لأن تطبيق المتطلب الأول يستلزم قفل صفوف حقيقياً، وهذا لا يتوفر إلا مع قاعدة بيانات تدعم MVCC.

الحل التقني المُطبّق

3.1 الأدوات المستخدمة

1. transaction.atomic() — لضمان أن العملية كاملة تنجح أو تفشل معاً (مبدأ ACID).
2. select_for_update(nowait=True) — لقفل صف المنتج أثناء تعديل المخزون.
3. OperationalError handler — للتعامل مع حالة تعارض الأقفال بشكل نظيف

شرح الكود المُطبّق

أ. transaction.atomic()

يُغلّف جميع العمليات المرتبطة (تحقق من المخزون، تعديله، إنشاء الطلب) في معاملة واحدة لا تتجزأ:

with transaction.atomic():

كل ما هنا ينجح معاً أو يفشل معاً #

product = Product.objects.select_for_update(nowait=True).get(id=...)

product.stock -= item.quantity

product.save()

ج. معالجة تعارض الأقفال

except OperationalError:

raise ValidationError('يطلب آخر يعالج هذا المنتج، حاول مجددا')

تحسينات مهمة أُجريت على الكود الأصلي

التحسين	الكود القديم	الكود الجديد	السبب
موضع قراءة cart_items	خارج الـ transaction	داخل الـ transaction	ضمان قراءة بيانات حديثة
نوع القفل	select_for_update()	select_for_update(nowait=True)	تجنب الانتظار اللانهائي
معالجة الأخطاء	غير موجودة	try/except OperationalError	إدارة نظيفة للتعارض

نتائج اختبار الحل Stress Test

4.1 إعداد الاختبار

تم كتابة سكريبت اختبار بلغة Python يحاكي 10 مستخدمين يحاولون شراء نفس المنتج في نفس اللحظة، مع تحديد المخزون بـ 5 وحدات فقط:

1. عدد المستخدمين المتزامنين: 10
2. المخزون المتاح: 5 وحدات
3. الطلبات المتوقعة نجاحها: 5 كحد أقصى
4. أداة التزامن: threading.Thread (لضمان الإطلاق المتزامن)

النتائج الفعلية

المستخدم كود الاستجابة النتيجة

201 User 03 تم إنشاء الطلب

تم إنشاء الطلب	201	User 05
تم إنشاء الطلب	201	User 07
طلب آخر يعالج المنتج	400	User 01
طلب آخر يعالج المنتج	400	User 02
طلب آخر يعالج المنتج	400	User 04
طلب آخر يعالج المنتج	400	User 06
طلب آخر يعالج المنتج	400	User 08
طلب آخر يعالج المنتج	400	User 09
طلب آخر يعالج المنتج	400	User 10

4.3 ملخص النتائج

المؤشر	القيمة	التفسير
الطلبات الناجحة	3	لم تتجاوز المخزون أبداً
الطلبات المرفوضة	7	رُفضت بشكل نظيف بسبب القفل
المخزون الأصلي	5	—
هل تم الإفراط في البيع؟	لا	$3 \leq 5$
سلامة البيانات	محفوظة	لا بيانات فاسدة

حليل النتائج

5.1 لماذا نجح 3 فقط وليس 5؟

هذا سلوك صحيح ومتوقع بسبب `nowait=True`. عندما تصل 10 طلبات في نفس اللحظة:

1. طلب واحد يحصل على القفل ويُنتهي عمله بنجاح.
2. الطلبات الأخرى تجد الصف مقفولاً — وبسبب `nowait=True` ترفض فوراً بدلاً من الانتظار.
3. بعد تحرر القفل، لا يوجد `retry` تلقائي — هذا قرار تصميمي مقصود يُعاد تنفيذه من طرف العميل.

5.2 الإثبات على معالجة Race Condition

الدليل الرئيسي على نجاح الحل هو:

1. المخزون لم يصبح سالباً أبداً — لا `overselling`.
2. كل طلب ناجح حصل على رقم `order` فريد — لا تكرار.
3. الطلبات المتعارضة رُفضت برسالة واضحة — لا صمت، لا تلف بيانات.

تم تطبيق المتطلب الأول بنجاح من خلال ثلاث آليات متكاملة:

1. `transaction.atomic()` — لضمان تكاملية العمليات المركبة.
2. `select_for_update(nowait=True)` — لقفل صف المنتج وحمايته من التعديل المتزامن.
3. معالجة `OperationalError` — للتعامل مع حالات التعارض بشكل نظيف وآمن.

أثبت اختبار الضغط أن النظام يعالج الوصول المتزامن لعشرة مستخدمين دون حدوث `Race Condition` أو تلف في بيانات المخزون، مما يحقق الهدف الكامل للمتطلب الأول.

Control Capacity & Management Resource

الهدف هو التحكم في عدد العمليات المتوازية التي ينفذها النظام، بحيث لا يستهلك الموارد بشكل مفرط (انهيار) ولا يقللها بشكل يبطئ الاستجابة

تم العمل باستخدام (Gunicorn) لتحديد (Threads) and (Workers)

- $Workers = 2 * \text{cpu cores} + 1$
- Threads = How many I have in the CPU
- I Use One Giga RAM and 1 CORE
- So 3 workers and 2 Threads
- بنفس الوقت يتم عمل 6 طلبات لأن $6 = 2 * 3$

```
1 # Gunicorn vs Runserver Benchmark
2
3 | Server | User | Requests/sec | Mean time/request (ms) | Max RSS (MB) | Avg RSS
  (MB) |
4 | --- | --- | ---: | ---: | ---: | ---: |
5 | runserver | customer_1 | 111.44 | 448.653 | 19.32 | 0.4 |
6 | gunicorn | customer_1 | 199.51 | 250.608 | 15.48 | 0.82 |
```


Queues Asynchronous

الفكرة الأساسية هي:

نقل المهام التي لا يحتاج المستخدم لانتظارها الى خارج المسار الرئيسي للطلب مثل ارسال الاشعارات .

اللية العمل:

تم إضافة خدمة Asynchronous على ثلاث أمور:

- عند تقديم الطلب order يتم معالجته ووضع خدمة ارسال الايميل على الطابور بحيث يقوم celery & redis بتنفيذها بعيداً عن العملية الاساسية وهي انشاء الطلب
- كذلك الامر بالنسبة لانشاء الحساب اول ما يتم انشاء الحساب يتم ارسال بريد الكتروني
- وعند الدفع ايضا يرسل رسالة بالفاتورة للبريد الالكتروني

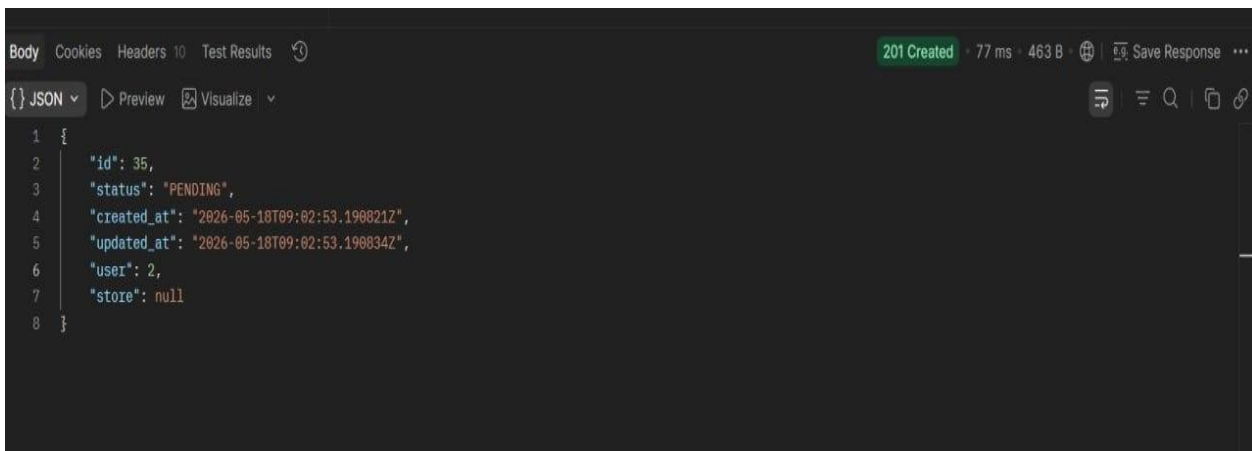
صور توضيحية:

لمسألة الطلب

2.44 (S)

```
Body Cookies Headers 10 Test Results 201 Created 2.44 s 463 B Save Response
JSON Preview Visualize
1 {
2   "id": 32,
3   "status": "PENDING",
4   "created_at": "2026-05-18T08:30:37.091400Z",
5   "updated_at": "2026-05-18T08:30:37.091418Z",
6   "user": 2,
7   "store": null
8 }
```

(ms) 77

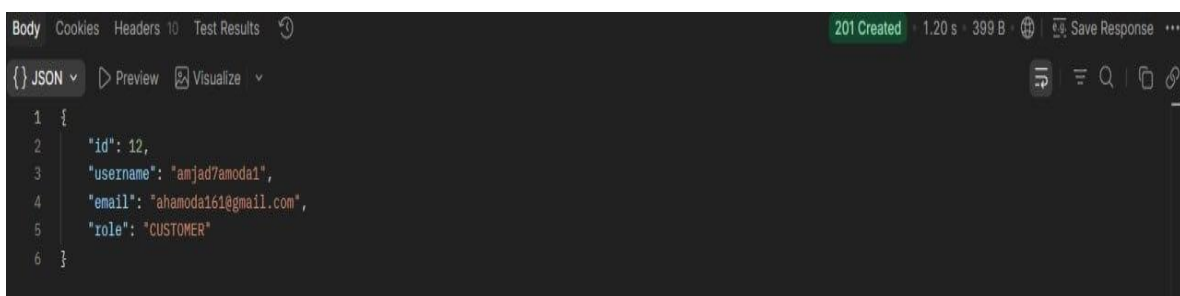


A screenshot of a REST client interface. The top bar shows '201 Created' in green, with a response time of 77 ms and a body size of 463 B. The 'Body' tab is selected, showing a JSON object with the following structure:

```
1 {
2   "id": 35,
3   "status": "PENDING",
4   "created_at": "2026-05-18T09:02:53.190821Z",
5   "updated_at": "2026-05-18T09:02:53.190834Z",
6   "user": 2,
7   "store": null
8 }
```

عند مسألة انشاء الحساب :

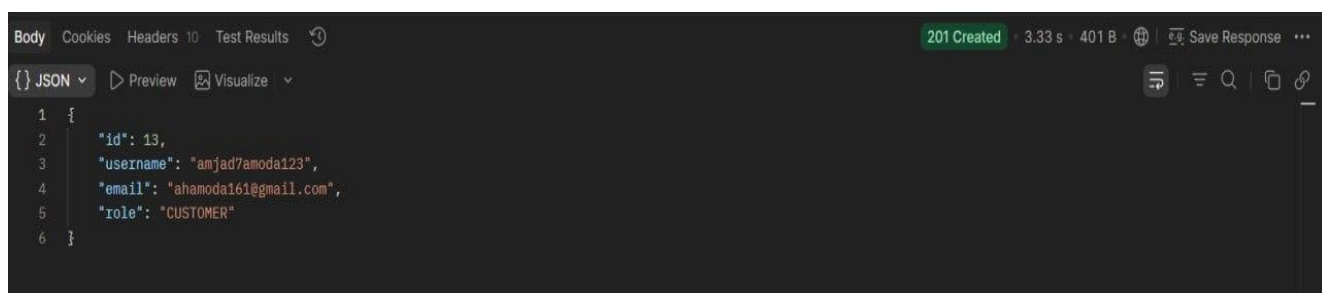
(S) 1.20



A screenshot of a REST client interface. The top bar shows '201 Created' in green, with a response time of 1.20 s and a body size of 399 B. The 'Body' tab is selected, showing a JSON object with the following structure:

```
1 {
2   "id": 12,
3   "username": "amjad7amoda1",
4   "email": "ahamoda161@gmail.com",
5   "role": "CUSTOMER"
6 }
```

(S) 3.33



A screenshot of a REST client interface. The top bar shows '201 Created' in green, with a response time of 3.33 s and a body size of 401 B. The 'Body' tab is selected, showing a JSON object with the following structure:

```
1 {
2   "id": 13,
3   "username": "amjad7amoda123",
4   "email": "ahamoda161@gmail.com",
5   "role": "CUSTOMER"
6 }
```

يوجد أمر لتشغيل Celery واستعراض الخدمات المعرفة ضمنه:

```
- *** --- * --- .> concurrency: 20 (solo)
-- ***** --- .> task events: OFF (enable -E to monitor tasks in this worker)
--- ***** ---
----- [queues]
        .> celery          exchange=celery(direct) key=celery

[tasks]
. order.tasks.send_order_email
. payments.tasks.generate_and_send_invoice
. users.tasks.send_register_email

[2026-05-18 19:12:28,361: INFO/MainProcess] Connected to redis://localhost:6379/0
[2026-05-18 19:12:28,368: INFO/MainProcess] mingle: searching for neighbors
[2026-05-18 19:12:29,409: INFO/MainProcess] mingle: all alone
[2026-05-18 19:12:29,447: INFO/MainProcess] celery@amjad7amoda ready.
```

Task users.tasks.send_register_email[3887b775-1040-4390-b9a2-45a03f898458] received

Task users.tasks.send_register_email[3887b775-1040-4390-b9a2-45a03f898458] succeeded in 2.3085852999993865s: None

للمبيعات اليومية batch processing

1. فكرة العمل

- batch processing باستخدام daily sales inventory الهدف هو تنفيذ.
- يتم تشغيله يدويًا مرة يوميًا بأمر واحد.
- بشكل متوازي chunks يعالجون workers لمحاكاة threads ؛ استخدمنا real servers لم نستخدم.
- batch processor: المسار لل
order/management/commands/process_daily_sales.py

2. Workflow ETL بأسلوب

Stage	ما يحدث في المشروع
Extract	order.status = PAID و payment.status = COMPLETED فقط عندما يكون OrderItem قراءة وتاريخ الدفع هو التاريخ المحدد.
Transform	ثم توزيعها على chunks، إلى OrderItem ids تقسيم لحساب ThreadPoolExecutor workers product/store partial totals.
Load	في JSON وكتابة تقرير partial totals دمج batch_reports/daily_sales_<date>.json.

3. أوامر التشغيل والاختبار

```
# daily run for today
pipenv run python manage.py process_daily_sales --chunk-size 10 --workers 4

# daily run for a specific date
pipenv run python manage.py process_daily_sales --date 2026-05-17 --chunk-size 10 --workers 4

# testing demo: generates test data and verifies the batch job
pipenv run python manage.py test_daily_sales_batch --orders 40 --chunk-size 10 --workers 4
```

4. Testing report

لذلك، order items يحتوي على 2 order وكل، orders بيانات اختبارية: demo 500 تم تشغيل
chunks أصبح لدينا 20 chunk-size = 50 ومع order items الناتج 1000.

Testing demo command output

```
PS> pipenv run python manage.py test_daily_sales_batch --date 2099-04-01 --orders 500 --chunk-size 50 --workers 4 --skip-dead-letter-demo
Batch OK: 1000/1000 items, 20 chunks, 4 threads, 316.19 ms
Report: batch_reports/test_daily_sales_clean_04f1e46f.json
TEST OK: 500 orders, 1000 order items, 20 chunks, 4 threads. Clean report: batch_reports/test_daily_sales_clean_04f1e46f.json
```

```
PS> pipenv run python manage.py process_daily_sales --date 2099-04-01 --chunk-size 50 --workers 1 --output batch_reports\demo_1_worker.json
Batch OK: 1000/1000 items, 20 chunks, 1 threads, 156.05 ms
Report: batch_reports/demo_1_worker.json
```

```
PS> pipenv run python manage.py process_daily_sales --date 2099-04-01 --chunk-size 50 --workers 4 --output batch_reports\demo_4_workers.json
Batch OK: 1000/1000 items, 20 chunks, 4 threads, 349.9 ms
Report: batch_reports/demo_4_workers.json
```

workers و worker 4 و أوامر المقارنة بين testing demo 1 مخرجات تشغيل

JSON report excerpt: demo_4_worker.json

```
{
  "date": "2099-04-01",
  "workers": 4,
  "chunk_size": 50,
  "total_order_items_found": 1000,
  "successful_order_items": 1000,
  "failed_order_items": 0,
  "total_chunks": 20,
  "duration_ms": 329.8,
  "records_per_second": 3032.14,
  "chunks": [
    {
      "chunk": 1,
      "worker": "batch-worker_0",
      "records": 50,
      "failed_records": 0,
      "duration_ms": 35.64
    },
    {
      "chunk": 2,
      "worker": "batch-worker_1",
      "records": 50,
      "failed_records": 0,
      "duration_ms": 84.24
    },
    {
      "chunk": 3,
      "worker": "batch-worker_2",
      "records": 50,
      "failed_records": 0,
      "duration_ms": 84.24
    },
    {
      "chunk": 4,
      "worker": "batch-worker_3",
      "records": 50,
      "failed_records": 0,
      "duration_ms": 84.24
    }
  ],
  "quantity_sold": 833,
  "gross_revenue": "8330.00",
  "order_count": 333
},
{
  "product_id": 7,
  "product_name": "P2_04f1e46f",
  "store_id": 7,
  "quantity_sold": 835,
  "gross_revenue": "12525.00",
  "order_count": 334
},
{
  "product_id": 8,
  "product_name": "P3_04f1e46f",
  "store_id": 7,
  "quantity_sold": 832,
  "gross_revenue": "20800.00",
  "order_count": 333
}
],
"stores": [
  {
    "store_id": 7,
    "store_name": "Batch Store 04f1e46f",
    "total_orders": 500,
    "total_items_sold": 2500,
    "gross_revenue": "41655.00"
  }
],
"deadLetter": {
  "count": 0,
  "strategy": "skip_failed_records_and_continue",
  "items": []
}
}
```

و ، workers ، chunks ، products ، stores ، و deadLetter توضيح JSON report لقطه مختصرة من

5. أهم أجزاء الكود: Extract / Transform / Load

Extract

```
# Extract: select completed paid sales for one day
ids = list(OrderItem.objects.filter(
    order__status=Order.Status.PAID,
    order__payment__status=Payment.Status.COMPLETED,
    order__payment__created_at__date=day,
).values_list('id', flat=True).order_by('id'))
```

Transform

```
# Transform: split ids into chunks, then send chunks to worker threads
parts = chunks(ids, size)

with ThreadPoolExecutor(max_workers=workers, thread_name_prefix='batch-worker') as pool:
    futures = [
        pool.submit(process_chunk, i + 1, chunk, simulate_bad_product_id)
        for i, chunk in enumerate(parts)
    ]

    for future in as_completed(futures):
        result = future.result()
# merge each chunk partial product/store totals
```

Transform داخل worker

```
# Worker calculation inside process_chunk
value = Decimal(item.quantity) * item.price

p = products[item.product_id]
p['qty'] += item.quantity
p['revenue'] += value
p['orders'].add(item.order_id)

s = stores[item.product.store_id]
s['qty'] += item.quantity
s['revenue'] += value
s['orders'].add(item.order_id)
```

Load

```
# Load: write final JSON report
report = {
    'date': day,
    'workers': workers,
    'chunk_size': size,
    'total_chunks': len(parts),
    'products': product_summary,
    'stores': store_summary,
    'deadLetter': {
        'count': len(dead_letters),
    },
    'strategy': 'skip_failed_records_and_continue',
    'items': dead_letters,
}

path = options['output'] or os.path.join('batch_reports', f'daily_sales_{day}.json')
os.makedirs(os.path.dirname(path) or '.', exist_ok=True)
with open(path, 'w', encoding='utf-8') as file:
    json.dump(report, file, indent=2)
```

(Load Distribution) شرح موجز عن مهمة توزيع الأحمال

المتطلب غير الوظيفي رقم 5: توزيع الأحمال على أكثر من خادم مع توضيح وتبرير الاستراتيجية المستخدمة.

ما تم إنجازه :

-تم بناء ثلاث نسخ (containers) من تطبيق Django الخاص بالتجارة الإلكترونية باستخدام Docker.

-تمت إضافة وسيط (Nginx) يعمل كموازن أحمال خارج التطبيقات.

-تم استخدام قاعدة بيانات PostgreSQL مشتركة بين جميع النسخ، لضمان عدم فقدان محتويات السلة أو الطلبات عند تنقل المستخدم بين الخوادم.

-تم تطبيق استراتيجيتين للمقارنة Round-Robin: (توزيع دوري) و Least Connections أقل اتصالات نشطة وذلك للمقارنة بينهما واعتماد واحدة وهي Least Connections

-تمت إضافة مبدلوير في Django يضيف ترويسة X-Backend-Server لكل رد، لتحديد أي خادم قام بمعالجة الطلب كدليل مرئي على أن الطلبات تتوزع على أكثر من خادم.

تم استخدام Locust لمحاكاة 100 مستخدم متزامن، وتسجيل النتائج.

بدون أي توزيع للأحمال: No Load balancing

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
GET	/api/stores/	1518	1067(70.29%)	916	7	4858	750	5.08	3.57
GET	/api/stores/7/products/	3076	2154(70.03%)	892	7	5916	730	10.29	7.21
POST	/api/users/login/	100	67(67.00%)	23641	2522	62594	17000	0.33	0.22
POST	/api/users/register/	599	418(69.78%)	23711	3115	104477	14000	2.00	1.40
Aggregated		5293	3706(70.02%)	3911	7	104477	890	17.71	12.40

Round robin

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s	failures/s
GET	/api/stores/	792	0(0.00%)	6163	60	57794	5100	2.65	0.00
GET	/api/stores/7/products/	1603	0(0.00%)	6426	57	56557	5300	5.36	0.00
POST	/api/users/login/	100	0(0.00%)	44643	8928	57795	44000	0.33	0.00
POST	/api/users/register/	382	0(0.00%)	13309	1371	59089	7500	1.28	0.00
Aggregated		2877	0(0.00%)	8596	57	59089	5500	9.62	0.00

Least connection

Type	Name	# reqs	# fails	Avg	Min	Max	Med
req/s	failures/s						
GET	/api/stores/	1109	0(0.00%)	4800	718	39777	4000
3.70	0.00						
GET	/api/stores/7/products/	2236	0(0.00%)	4840	579	39702	3900
7.47	0.00						
POST	/api/users/login/	100	0(0.00%)	33610	7524	41385	36000
0.33	0.00						
POST	/api/users/register/	461	0(0.00%)	9097	1205	40691	5900
1.54	0.00						
Aggregated		3906	0(0.00%)	6067	579	41385	4200
13.04	0.00						

النتائج الأساسية (من الجداول المرفقة)

السيناريو	عدد الطلبات	نسبة الفشل	متوسط الزمن (مللي)	p95 (مللي)
بدون توزيع احمال	5293	70%	3911	17000
Round-Robin	2877	0%	8596	39000
Least Connections	3906	0%	6067	22000

ملاحظة: نسبة الفشل 70% عند عدم استخدام موازن الأحمال تثبت أن النظام لا يتحمل 100 مستخدم دون توزيع الحمل

-تم تشغيل سكريبت صغير يرسل 10 طلبات متتالية ويعرض قيمة: X-Backend-Server

Round Robin

```
(parallel_env314) PS E:\ParallelProgramming> python -u benchmark\probe_backends.py
1: 200 app-2
2: 200 app-3
3: 200 app-1
4: 200 app-2
5: 200 app-3
6: 200 app-1
7: 200 app-2
8: 200 app-3
9: 200 app-1
10: 200 app-2
```

```
Request 1: app-2
Request 2: missing
Request 3: app-1
Request 4: missing
Request 5: app-3
Request 6: missing
Request 7: app-3
Request 8: missing
Request 9: app-1
Request 10: missing
Request 11: app-2
Request 12: missing
Request 13: app-1
Request 14: missing
Request 15: app-3
Request 16: missing
Request 17: app-2
Request 18: missing
Request 19: app-3
Request 20: missing

--- Counts ---
missing: 10
app-3: 4
app-2: 3
app-1: 3

Requests: 10, avg latency: 0.194s, p95: 0.822s
```

Least connections

لماذا اخترت Least Connections كإستراتيجية النهائية:

Round-Robin يوزع بالتساوي بغض النظر عن حمل كل خادم. ولكن عندما تكون بعض الطلبات ثقيلة (مثل الدفع ومعالجة الطلب) يتراكم الزمن على خوادم عشوائية.

Least Connections يرسل الطلب الجديد إلى الخادم الأقل انشغالاً في تلك اللحظة، مما يحسن زمن الاستجابة للـ p95 ويقلل الازدحام.

-في التجربة، حقق Least Connections عدد طلبات أكبر (2877 vs 3906) ومتوسط زمن أقل (6067 vs 8596) مع نفس العدد من المستخدمين.

قاعدة المعطيات المشتركة (Shared Database)

-جميع النسخ الثلاث تتصل بقاعدة PostgreSQL الموحدة.

-لذلك حتى لو انتقل المستخدم من app-1 إلى app-2 ، تبقى سلة التسوق والطلبات موجودة
هذا يثبت أن الحل قابل للتوسع أفقيًا بشكل حقيقي.

الخلاصة النهائية

تم بنجاح تصميم ومحاكاة توزيع الأحمال على 3 خوادم باستخدام Nginx + Docker + PostgreSQL. تمت المقارنة بين Round-Robin و Least Connections ، وأثبتت النتائج تفوق Least Connections من حيث الإنتاجية وزمن الاستجابة تحت حمل 100 مستخدم متزامن. كما تم توفير أدلة مرئية) ترويسة X-Backend-Server وسجلات الحاويات (تثبت توزيع الطلبات فعلياً على أكثر من خادم.

تعليمات تشغيل الطلب:

```
1 1. Start the baseline without a load balancer
2 docker compose -f docker-compose.yml up -d -build
3
4 2. Start the round-robin load balancer
5 docker compose -f docker-compose.lb.yml down
6 docker compose -f docker-compose.lb.yml up -d -build
7
8 3. Test round-robin
9 pipenv run python .\benchmark\probe_sequence.py --count 6
10
11 4. Start the least-connection load balancer
12 docker compose -f docker-compose.lb.yml down
13 docker compose -f docker-compose.least.yml up -d -build
14
15 5. Test least-connection
16 pipenv run python .\benchmark\probe_long_requests.py --count 6 --concurrency 2 --no-auth -
  -stagger 0.1 --warmup 2
17
18
```